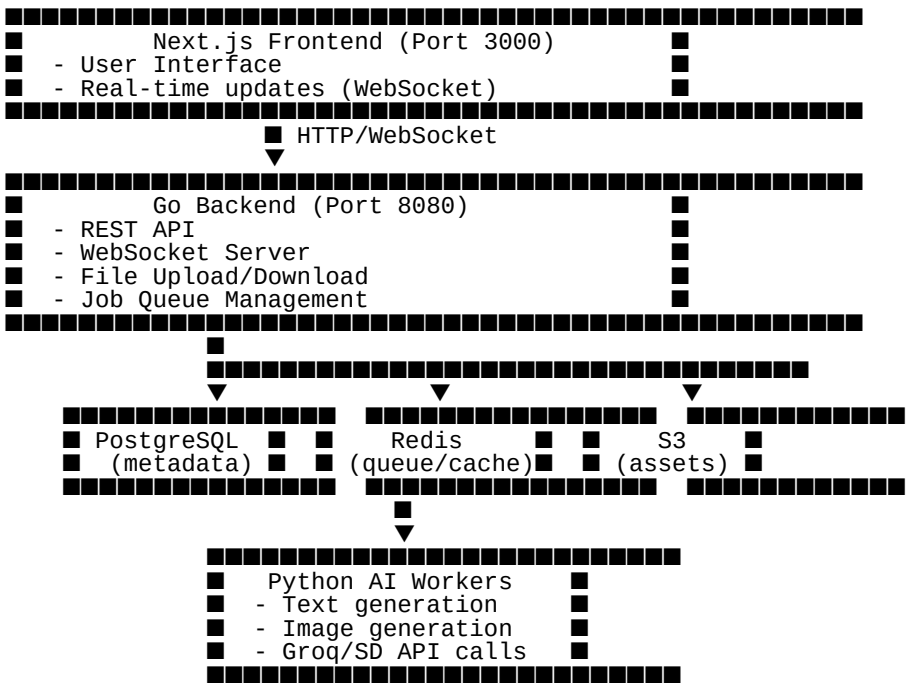
Creative Automation Hub - Architecture

System Architecture



Component Details

Go Backend Responsibilities

API Gateway: Handle all HTTP requests WebSocket: Real-time progress updates File Management: Upload/download/S3 Job Queue: Distribute AI tasks to workers Authentication: JWT tokens Rate Limiting: Prevent abuse Python AI Workers

Text Generator: Groq/Claude integration Image Generator: Stable Diffusion/DALL-E Workers: Process Redis queue jobs Isolated: Can scale independently Next.js Frontend

SSR: Fast initial load Real-time: WebSocket updates Canvas Editor: Fabric.js integration State Management: React Context/Zustand Data Flow

Content Generation Flow

```
1. User clicks "Generate" in Next.js
2. Next.js → POST /api/generate → Go Backend
3. Go creates job → Redis Queue
4. Python Worker picks job → Calls AI API
5. Worker saves result → S3 + PostgreSQL
6. Go sends WebSocket update → Next.js
7. User sees generated content
```

Scaling Strategy

Phase 1: Single Server
- Go + Python workers on same machine
- Local Redis + PostgreSQL

Phase 2: Horizontal Scaling
- Multiple Python workers
- External Redis/PostgreSQL
- Load balancer for Go instances

Phase 3: Microservices
- Separate services for text/image
- Kubernetes deployment
- Auto-scaling based on queue depth

Technology Choices

Why Go for Backend:
- Handles 10K+ concurrent connections
- Built-in concurrency (goroutines)
- Fast JSON processing
- Low memory footprint
- Easy deployment (single binary)

Why Python for AI:
- Best ML library support
- Groq/OpenAI SDKs
- Stable Diffusion ecosystem
- Easy to prototype

Why Next.js:
- Best React framework

- Built-in API routes
- Image optimization
- Fast development

Performance Targets

API response: < 50ms WebSocket latency: < 100ms Text generation: 2-5 seconds Image generation: 5-15 seconds Concurrent users: 1000+ Batch jobs: 100 simultaneous Security